



PROCESADORES DE LENGUAJE - P3 y P4

Jorge Mejías Donoso 100495807@alumnos.uc3m.es

Alberto Menchen Montero 100495692@alumnos.uc3m.es

PREGUNTAS ABIERTAS P3:

3.1 Adapta la gramática para que las expresiones encerradas con paréntesis puedan ir precedidas de un signo negativo (o positivo). Indicar sólo las producciones nuevas necesarias. Se recomienda probar la solución con bison. ¿Funciona para la expresión $-(-3)$?

Anteriormente la expresión $-(-3)$ no es reconocida y se devuelve un syntax error por lo que para solucionarlo hemos modificado la gramática de la siguiente manera.

```
operando:      resto_operando      { $$ = $1; }
            | '-' resto_operando    { $$ = -$2; }
            | '+' resto_operando    { $$ = $2; }
            ;

resto_operando:
            | NUMERO                { $$ = $1; }
            | '(' expresion ')'     { $$ = $2; }
            ;
```

Para adaptar la gramática y permitir que las expresiones encerradas entre paréntesis puedan ir precedidas de un signo negativo o positivo, se han añadido nuevas producciones en la regla operando. En el caso propuesto $-(-3)$, la producción operando: `'-' resto_operando { $$ = -$2; }` permite que un signo negativo anteceda cualquier resto_operando, incluyendo expresiones entre paréntesis. Como resto_operando ahora reconoce `(' expresion ')`, esto garantiza que expresiones como $-(-3)$ sean correctamente interpretadas. Cuando se analiza $-(-3)$, primero se reconoce (-3) como un resto_operando, evaluando su contenido a -3 . Luego, la producción `'-' resto_operando` aplica el signo negativo a -3 , resultando en 3 . Por lo tanto, la adaptación realizada en la gramática permite procesar correctamente este tipo de expresiones y asegura que la calculadora maneje los signos de manera adecuada.

3.2 Adapta la gramática para reconocer números reales (ejemplos: 00.33, 0.33, .33, 100, 0., 100.33). Representa las nuevas producciones de la gramática en BNF y en forma de Diagrama Sintáctico. No se pide ni código C ni acciones semánticas. Tampoco se pide diferenciar nivel léxico o sintáctico, sólo incluir las producciones necesarias para reconocer números reales (como los indicados como ejemplo).

Nuestra solución adapta la gramática para reconocer números reales dividiéndolos en tres componentes principales: la parte entera, el punto decimal y la parte decimal. La parte entera, representada por `'n_entero'`, puede estar formada por un solo dígito o por una

secuencia de dígitos. La parte decimal, representada por 'n_decimal', también puede contener uno o más dígitos después del punto. La regla 'numero' combina estos elementos, asegurando que cualquier número real válido incluya un punto decimal entre la parte entera y la decimal. Gracias a esta estructura, la gramática permite reconocer distintos formatos de números, como '100.33', '0.33', '.33', '100.' y '00.33', cubriendo así todas las variantes solicitadas en la pregunta.

```
numero:      n_entero '.' n_decimal { $$ = $1 + $3 ; }
           | '.' n_decimal          { $$ = $2 ; }
           | n_entero                { $$ = $1 ; }
           ;

n_entero:     DIG                    { $$ = $1 ; pot = 1 ; }
           | DIG n_entero            { pot *= 10 ; $$ = $1 * pot + $2 ; }
           ;

n_decimal    : DIG                    { $$ = $1 / 10.0 ; }
           | DIG n_decimal            { $$ = ($1 + $2) / 10.0 ; }
           ;
```

También hemos tenido que añadir en la cabecera del archivo las líneas 'double pot' y '%token DIG'.

```
%{
#include <stdio.h>
#define YYSTYPE double
double pot;
%}
%token DIG
```

3.3 Revisa la gramática de calc3.y (representala con diagramas sintácticos) y contrástala con el diagrama diag3. ¿Ves alguna diferencia, incluso sutil? ¿Los lenguajes generados o reconocidos por ambas representaciones difieren en algo? ¿Ves algún problema en implementar el analizador sintáctico, junto con su semántica siguiendo el diagrama?

Las principales diferencias entre la representación en diagrama y la gramática escrita están en cómo se organizan las operaciones y los signos. Mientras que en el diagrama los operadores '+', '-', '*' y '/' se agrupan en una única categoría llamada "operador", en la gramática se definen como reglas separadas. Lo mismo ocurre con los signos '+' y '-', que en el diagrama se combinan bajo una sola caja denominada "signo", pero en la gramática aparecen en producciones distintas.

Otra diferencia importante es la forma en que se maneja el axioma. En el código, se introduce un no terminal auxiliar ('r_expr') para evitar problemas de ambigüedad cuando se incorpora la semántica, mientras que en el diagrama no es necesario este ajuste. Esto

significa que, aunque visualmente el diagrama parece correcto, en la implementación práctica se requieren modificaciones en la gramática para garantizar su correcto funcionamiento.

Además, aunque en teoría se podría unificar los operadores bajo un único no terminal ('Operador') para simplificar la gramática, en la práctica esto complicaría la aplicación de la semántica, ya que cada operación requiere un tratamiento específico en el cálculo de las expresiones.

PREGUNTAS ABIERTAS P4:

4.1 Prueba las siguientes expresiones para comprobar el funcionamiento de la precedencia. ¿a qué conclusión llegas? $2*3+1$ $2+3*1$ $1+3*2$ $1*3+2$

El código no es capaz de identificar la preferencia que tiene cada uno de los signos. Con la nueva solución implementada el parser respeta la precedencia de operadores tal como está definida en Bison:

- Las multiplicaciones y divisiones se resuelven antes que sumas y restas.
- Se evalúa de izquierda a derecha en caso de operadores con la misma precedencia.
- El parser no requiere paréntesis adicionales para respetar la jerarquía estándar de operadores.

4.2 Reforma la gramática sustituyendo el no terminal operando por el de expresion en aquellas producciones que contengan una operación binaria. Recuerda que habíamos introducido operando para evitar la ambigüedad debida a la doble recursividad (expresionexpresion+expresion). Prueba de nuevo las expresiones anteriores.

La modificación del código correspondiente es la siguiente:

```
expresion:      expresion '+' expresion { $$ = $1 + $3; }
               | expresion '-' expresion { $$ = $1 - $3; }
               | expresion '*' expresion { $$ = $1 * $3; }
               | expresion '/' expresion { $$ = $1 / $3; }
               | NUMERO                  { $$ = $1; }
               | '+' expresion %prec SIGNO_UNARIO { $$ = $2; }
               | '-' expresion %prec SIGNO_UNARIO { $$ = -$2; }
               | '(' expresion ')'         { $$ = $2; }
               ;

%%
```

Se ha eliminado el no terminal operando y se ha sustituido por expresión en las producciones que contienen operaciones binarias. Originalmente, operando se introdujo para evitar ambigüedades debido a la doble recursividad en expresiones del tipo:

expresion → expresion + expresion

Con esta modificación, la gramática sigue funcionando sin errores ni conflictos, garantizando que los operadores se evalúan en el orden correcto. Las pruebas realizadas con todos los ejemplos anteriores corroboran la evidencia de funcionamiento.

4.3 Si nos plantearan usar variables como operandos dentro del lenguaje, ¿qué cambios plantearías en la gramática? Partimos de la idea de que las variables estarán identificadas por una secuencia alfanumérica, siendo el primer carácter una letra. No se contempla la definición previa de variables.

%token NUMERO VARIABLE

Nuevo token VARIABLE: Se define un nuevo token VARIABLE para reconocer las variables en el lenguaje.

```
expresion:    expresion '+' expresion { $$ = $1 + $3; }
              | expresion '-' expresion { $$ = $1 - $3; }
              | expresion '*' expresion { $$ = $1 * $3; }
              | expresion '/' expresion { $$ = $1 / $3; }
              | NUMERO                  { $$ = $1; }
              | VARIABLE                 { $$ = 0; /* Placeholder value for variables */ }
              | '+' expresion %prec SIGNO_UNARIO { $$ = $2; }
              | '-' expresion %prec SIGNO_UNARIO { $$ = -$2; }
              | '(' expresion ')'         { $$ = $2; }
              ;
```

Gramática modificada: La regla expresión se modifica para incluir VARIABLE como un posible operando. En este ejemplo, las variables se tratan con un valor de marcador de posición (0) ya que no se contempla la definición previa de variables.

```

int yylex()
{
    unsigned char c;
    do {
        c = getchar();
    } while (c == ' ');

    if (c == '.' || (c >= '0' && c <= '9')) {
        ungetc(c, stdin);
        scanf("%lf", &yylval);
        return NUMERO;
    }

    if ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')) {
        /* Identifica una variable */
        while ((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') ||
            (c >= '0' && c <= '9')) {
            c = getchar();
        }
        ungetc(c, stdin);
        return VARIABLE;
    }

    if (c == '\n')
        n_linea++;

    return c;
}

```

Analizador léxico (yylex): Se actualiza para reconocer secuencias alfanuméricas que comienzan con una letra como variables. Las variables se reconocen y se devuelve el token VARIABLE.

4.4 Incluye la operación de asignación de una expresión a una variable en la gramática.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define YYSTYPE double

#define MAX_VARIABLES 256

double variables[MAX_VARIABLES]; // Array para almacenar valores de variables (a-z)
char current_variable; // Variable global para almacenar el nombre de la variable actual

int get_variable_index(char var) {
    return var - 'a';
}

```

Definición de MAX_VARIABLES: Se define el tamaño máximo del array de variables.

Array variables: Se declara un array variables para almacenar los valores de las variables (a-z).

Variable current_variable: Se declara una variable global current_variable para almacenar el nombre de la variable actual.

Función get_variable_index: Se define una función para obtener el índice de una variable en el array variables.

```

axioma:      expresion '\n' { printf("Expresion=%lf\n", $1); } r_expr
| VARIABLE '=' expresion '\n' {
    variables[get_variable_index(current_variable)] = $3;
    printf("Asignacion: %c = %lf\n", current_variable, $3);
} r_expr
;

r_expr:      /* lambda */
| axioma
;

```

VARIABLE '=' expresion '\n': Esta parte de la regla axioma se activa cuando se encuentra una variable seguida del operador de asignación (=), una expresión y un salto de línea (\n).

{ variables[get_variable_index(current_variable)] = \$3;: La acción asociada a esta regla asigna el valor de la expresión a la variable. current_variable contiene el nombre de la variable actual, y get_variable_index(current_variable) obtiene el índice correspondiente en el array variables.

r_expr: Esta parte permite la recursividad, permitiendo que se puedan realizar múltiples asignaciones en secuencia, conservando el valor de las variables y pudiendo operar con sus valores almacenados.

La función **yylex**:

Reconocer números: Si encuentra un número (incluyendo decimales), lo lee y lo devuelve como token NUMERO.

Reconocer variables: Si encuentra una letra (a-z o A-Z), la almacena en `current_variable` y la devuelve como token `VARIABLE`.

4.5 Intenta adaptar el analizador lexicográfico correspondiente.

Para implementar en analizador lexico junto con el semantico hemos delegado ciertas tareas al lexico como son el reconocimiento de números, variables, espacios en blanco, nuevas líneas y literales. Esto permite que el analizador sintáctico (``calc4.y``) se enfoque en las reglas de producción y la lógica de la gramática, mientras que el analizador léxico se encarga de la tokenización de la entrada. Se explican a continuación.

Ignorar espacios y tabuladores:

Se ignoran los espacios y tabuladores para que no interfieran con el análisis sintáctico.

```
[ \t]          { ; } /* ignorar espacios y tabuladores */
```

Reconocimiento de números:

Se reconocen números enteros y reales utilizando expresiones regulares. Los números se convierten a tipo `'double'` y se asignan a `'yyval'`.

```
[0-9]+(\.[0-9]*)?|\.[0-9]+ { sscanf(yytext, "%lf", &yyval); return NUMERO; }
```

Reconocimiento de variables:

Se reconocen variables que son caracteres alfabéticos `'(a-z o A-Z)'`. El carácter se almacena en `'current_variable'` y se asigna a `'yyval'`.

```
[a-zA-Z]      { current_variable = yytext[0]; return VARIABLE; }
```

Contar nuevas líneas:

Se cuenta el número de líneas para el manejo de errores.

```
\n          { n_linea++; return '\n'; }
```

Reconocimiento de literales:

Se reconocen otros caracteres literales y se devuelven tal cual.

```
.          { return yytext[0]; } /* literales */
```

4.6 ¿Qué problema ves a la hora de implementar un analizador sintáctico que sea totalmente operativo (a nivel semántico) con las últimas cuestiones propuestas?

Para que el analizador sintáctico sea totalmente operativo a nivel semántico, se necesitarían mejoras en el manejo de variables, la gestión de errores semánticos, la capacidad de manejar expresiones más complejas, y la extensibilidad para futuras características. Además, sería beneficioso implementar optimizaciones para mejorar la eficiencia del análisis y la evaluación de expresiones.

